EduStack Masterclass

Your Ultimate Computer Science & Engineering Hub

VOLUME 1 — Node.js Internals, Backend Architecture & Systems

Tier-1 Interview Reference Guide (Visa, Amazon, Oracle, JPMC, Microsoft)

V8 Engine | Libuv Event Loop | Express Pipeline | Error Architecture | 25 Tier-1 Q&As

Developer:	Shubham Kumar | CSE Student | NIT Patna
Target Roles:	Software Development Engineer (SDE II/III), Backend Architect
Core Focus:	Node.js Event Loop, V8 Engine, Thread Pool, Express Middleware, Error Bounds
Architecture:	Monolith + Microservice Hybrid (Node.js 18 + Python 3.11 FastAPI)
Database:	MongoDB Atlas + Mongoose ODM (Connection Pooling, Indexing)
Deployment:	Render.com Web Service + Reverse Proxy Trust + Graceful Shutdown
Volume 1:	Node.js Runtime Internals, Express Pipeline, Error Handling & 25 Q&As

github.com/ShubhamKumar968/EduStack--Your-Ultimate-Computer-Science-Hub

For SDE Technical Interview Preparation — Volume 1 of 4

Table of Contents — Volume 1

Masterclass Note: Volume 1 provides foundational backend engineering depth. Volume 2 covers Auth & Security, Volume 3 covers Databases & Cloud, and Volume 4 covers ML Engine & System Design Scenarios.

SECTION 1

Enterprise Problem Statement & Blueprint

Why EduStack was engineered to demonstrate senior-level backend capabilities

1.1 Project Vision & The Problem Space

EduStack was architected to solve a systemic problem in engineering education: academic materials, previous year examination papers, and interview preparation trackers are fragmented across disparate, unindexed platforms. Students waste valuable engineering hours searching across Telegram channels, Reddit drives, and random repositories.

Rather than building a basic CRUD application, EduStack was designed as an enterprise-grade platform incorporating stateless security, microservice proxying, cloud media pipelines, live data synchronization, and automated AI tutoring. The architecture reflects production requirements for high availability, fault tolerance, and security compliance.

Key Module Architectures

- **Subject Core Engine**: Manages 42+ computer science subjects categorized across 8 semesters. Built on MongoDB Atlas with Mongoose schema validation, indexes on semester and code, and virtual population.
- **AI Tutor & Multimodal Engine**: Microservice architecture powered by Python FastAPI, Google Gemini 1.5/2.0 Flash, and LightRAG (Retrieval-Augmented Generation). Provides grounded course Q&A and PDF document summarization.
- **Premium DSA Competitive Tracker**: 450+ hand-curated competitive programming problems synced live from a published Google Sheet CSV with an in-memory 5-minute TTL cache and disk fallback.
- **Stateless Security Engine**: Dual auth system (Local Email/Password + Google OAuth 2.0) issuing JWTs stored exclusively in httpOnly, Secure, SameSite cookies.
- **Transactional Payment Pipeline**: Razorpay gateway integration featuring server-side order initialization and HMAC-SHA256 constant-time signature verification.

SECTION 2

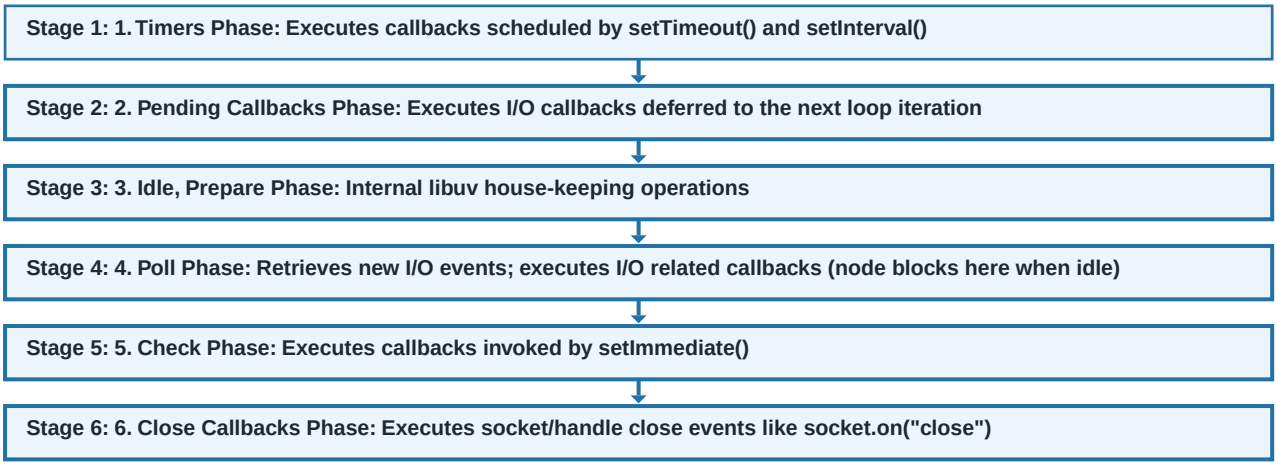
Node.js Runtime & V8 Engine Internals

Deep dive into the single-threaded event loop, libuv thread pool, and execution phases

2.1 The Node.js Event Loop Architecture

Node.js operates on a single-threaded event-driven architecture powered by Google V8 and libuv. Understanding how Node handles concurrent I/O without multi-threading is crucial for tier-1 system design and backend engineering interviews.

ARCHITECTURE MAP: Libuv Event Loop Execution Phases



Microtask Queue vs Macrotask Queue

Inside Node.js, asynchronous operations are divided into Microtasks and Macrotasks. Microtasks have higher priority and run immediately after the current operation completes, before the event loop moves to the next phase:

- **Microtask Queue**: process.nextTick() queue has top priority, followed by the Promise microtask queue (.then(), async/await). All microtasks are exhausted completely before moving to the next event loop phase.
- **Macrotask Queue**: Includes setTimeout, setInterval, setImmediate, and I/O callbacks. They are processed phase-by-phase according to libuv scheduling.

FAANG INTERVIEW TIP: When asked "What is the difference between process.nextTick and setImmediate?": "process.nextTick fires immediately after the current operation finishes, BEFORE the event loop continues to any phase. setImmediate fires in the Check phase of the event loop. Overusing process.nextTick can starve the event loop by preventing I/O polling."

JavaScript / Node.js Production Implementation

```javascript
// Demonstrating Event Loop Order in Node.js
console.log('1. Synchronous Start');

setTimeout(() => console.log('2. setTimeout (Timers Phase)'), 0);
setImmediate(() => console.log('3. setImmediate (Check Phase)'));

process.nextTick(() => console.log('4. process.nextTick (Microtask)'));
Promise.resolve().then(() => console.log('5. Promise.then (Microtask)'));

console.log('6. Synchronous End');

// Output Order:
// 1. Synchronous Start
// 6. Synchronous End
// 4. process.nextTick (Microtask - Highest Priority)
// 5. Promise.then (Microtask)
// 2. setTimeout (Timers Phase)
// 3. setImmediate (Check Phase)
```

SECTION 3

System Architecture & Hybrid Design

Monolith vs Microservices trade-offs, network boundaries, and HTTP proxying

3.1 Architectural Trade-offs: Why Monolith + Microservice?

EduStack implements a hybrid monolith-microservice architecture. The main application is a monolithic Express.js server handling REST APIs, authentication, and database interactions. The AI processing engine is isolated into an independent Python FastAPI microservice.

- **Node.js Monolith Benefits**: Low operational overhead, zero inter-service network latency for core CRUD, single deployment pipeline, simplified database connection management.
- **Python FastAPI Microservice Benefits**: Access to Python's native AI/ML libraries (Google Gemini SDK, pypdf, numpy), independent CPU scaling for AI workloads without blocking Express I/O, independent deployment lifecycle.
- **HTTP Proxy Pattern**: Node.js acts as an authenticated proxy. The browser never communicates directly with the Python FastAPI service, maintaining centralized JWT authentication and rate limiting in Node.js.

SECTION 4

Express.js Middleware Pipeline Deep Dive

Exact registration order, arity of error handlers, and reverse proxy trust configuration

4.1 The Middleware Pipeline in Production

Express processes requests sequentially through its middleware chain. The registration order determines security enforcement, body parsing, and request logging. Missing or misordered middleware can introduce severe vulnerabilities.

Order	Middleware	Exact Function & Security Rationale
1	helmet()	Attaches 15+ HTTP security headers. Must run first to secure all responses.
2	cors()	Evaluates Origin header against whitelist. Handles OPTIONS preflight requests.
3	trust proxy	Configures Express to trust X-Forwarded-For headers from Render/AWS load balancers.
4	express.json()	Parses JSON request bodies. Configured with 1MB limit to prevent memory DoS.
5	cookieParser()	Parses Cookie header into req.cookies. Required before isAuth reads JWT cookies.
6	morgan()	Logs request method, URL, status code, and response latency for audit logs.
7	mongoSanitize()	Strips MongoDB query operators ($gt, $where) to prevent NoSQL injection.
8	session() + passport	Manages OAuth session state required for Google OAuth 2.0 consent flow.
9	API Routes	Dispatches request to controller handlers (e.g., /api/auth, /api/subjects).
10	errorHandler	Global 4-parameter error handler. Catches all unhandled exceptions via next(err).

JavaScript / Node.js Production Implementation

```
// Production Express Middleware Pipeline Initialization
const express = require('express');
const helmet = require('helmet');
const cors = require('cors');
const cookieParser = require('cookie-parser');
const mongoSanitize = require('express-mongo-sanitize');
const morgan = require('morgan');

const app = express();

// 1. Security Headers
app.use(helmet());

// 2. CORS Allowlist
app.use(cors({
  origin: (origin, cb) => {
    const allowed = (process.env.CORS_ORIGINS || '').split(',');
    if (!origin || allowed.includes(origin)) return cb(null, true);
    cb(new Error('CORS Policy Violation'));
  },
  credentials: true,
}));

// 3. Trust Reverse Proxy (Render.com / AWS ALB)
if (process.env.NODE_ENV === 'production') {
  app.set('trust proxy', 1);
}

// 4. Body Parsing & Cookie Parsing
app.use(express.json({ limit: '1mb' }));
app.use(cookieParser());
app.use(mongoSanitize());
app.use(morgan('combined'));
```

SECTION 5

Global Error Architecture & Async Handling

Centralized exception boundaries, operational vs programmer errors, and status mapping

5.1 The asyncHandler Wrapper Pattern

In Express 4, unhandled promise rejections inside async route handlers do not automatically reach the global error handler - they result in unhandled promise rejections that can hang requests or crash the node process. EduStack uses an higher-order asyncHandler function to automatically capture rejected promises and forward them via next(err).

JavaScript / Node.js Production Implementation

```
// Higher-Order Async Handler Function
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

// Centralized 4-Parameter Error Middleware
const errorHandler = (err, req, res, next) => {
  let status = err.statusCode || 500;
  let message = err.message || 'Internal Server Error';

  // Map Mongoose Validation Errors
  if (err.name === 'ValidationError') {
    status = 400;
    message = Object.values(err.errors).map(e => e.message).join(', ');
  }
  // Map MongoDB Duplicate Key Violation (E11000)
  if (err.code === 11000) {
    status = 409;
    message = `Duplicate entry for ${Object.keys(err.keyValue)[0]}`;
  }
  // Map JWT Errors
  if (err.name === 'JsonWebTokenError') { status = 401; message = 'Invalid Token'; }
  if (err.name === 'TokenExpiredError') { status = 401; message = 'Session Expired'; }

  res.status(status).json({
    success: false,
    message,
    errors: process.env.NODE_ENV === 'development' ? [err.stack] : [],
  });
};
```

SECTION 6

Tier-1 Interview Q&A - Node.js & Backend

25 Deep Technical Questions asked by Visa, Amazon, Oracle, JPMC, Microsoft

Q: Explain how Node.js handles 10,000 concurrent HTTP requests with a single thread.

Comprehensive Technical Answer:

Node.js delegates non-blocking network I/O operations to the operating system via libuv event demultiplexers (epoll on Linux, kqueue on macOS, IOCP on Windows). The single main thread registers callbacks and immediately continues processing other events. When network data arrives, libuv pushes the callback into the event loop queue. Because network I/O involves waiting rather than CPU computation, one thread can easily manage 10,000 active socket descriptors.

-> V8 executes JavaScript synchronous code on the single call stack.

-> libuv manages the event loop and background thread pool for file/crypto I/O.

-> Memory overhead per connection is minimal (~2KB per socket) compared to OS thread-per-request models (1MB per thread in Java).

Q: What is the difference between CPU-bound and I/O-bound tasks in Node.js?

Comprehensive Technical Answer:

I/O-bound tasks (database queries, network requests, disk reads) spend most time waiting for external hardware or network devices. Node.js excels at I/O-bound tasks due to its asynchronous non-blocking model. CPU-bound tasks (image processing, video encoding, complex crypto, heavy algorithms) block the single main thread, preventing the event loop from processing any other incoming requests. CPU tasks should be offloaded to Worker Threads or external microservices.

-> EduStack offloads heavy AI/PDF operations to a Python FastAPI microservice.

-> Worker Threads (worker_threads module) can be used for CPU tasks in Node.js without blocking main event loop.

Q: What happens if an uncaught exception occurs inside an async function in Node.js?

Comprehensive Technical Answer:

An uncaught exception inside an async function returns a rejected Promise. If that Promise has no .catch() block or is not wrapped in asyncHandler to pass the error to next(err), it triggers an "unhandledRejection" event on the process object. In modern Node.js versions, unhandled rejections terminate the process with non-zero exit code if unhandled.

-> Always wrap async controllers with asyncHandler or try/catch forwarding to next(err).

-> Attach process.on("unhandledRejection") listener to gracefully close HTTP servers before exiting.